

MyTech v2 FastGPT

Functional Test Report

Version scope: MyTech v2 (FastGPT integration)

Report date: 21 September 2026

Overall result: PASS for classroom submission/demo

Core recommendation, follow-up, multi-product, clarification, scope routing, Restart, product-card rendering, and cross-platform local execution were verified. A final read-only code QA identified only non-blocking robustness items.

Important evidence note: most core functional regression tests were completed before the later model switch to DeepSeek. After the model switch, targeted help/greeting intent checks were re-run. A full post-switch regression was intentionally not repeated to conserve platform quota.

1. Executive Summary

Purpose. This report consolidates the MyTech v2 functional tests recorded during the FastGPT migration and live integration work. It is intended as a concise verification record for the classroom project and group testing preparation.

System under test. MyTech v2 uses the existing Editorial Clean frontend, a Node.js backend, and a published FastGPT workflow. The frontend and backend communicate through the same-origin /api/mytech route. The FastGPT API key remains server-side.

Result. The recorded manual tests passed for the core user journeys. A final Codex read-only QA also judged the package ready for final classroom submission and demonstration, with two low-severity/non-blocking robustness observations documented later in this report.

2. Test Scope and Environment

Area	Recorded scope
Supported categories	Laptop, Desktop, Smartphone, Tablet, Headphones, Monitor, Smartwatch, Mouse, Keyboard.
Approved dataset	27 products across the nine supported categories.
Frontend	Editorial Clean UI with structured response rendering and product cards.
Backend	Node.js server using environment variables for FastGPT configuration.
Conversation design	Stable per-browser chatId during a session; Restart rotates chatId and clears local state.
Platforms verified	macOS and Windows local execution.
Evidence types	Manual live tests, FastGPT preview tests, real MyTech-to-FastGPT integration tests, and read-only code QA.

3. Manual Functional Test Matrix

ID	Test scenario	Expected/verified behavior	Result
T01	Live FastGPT integration	Browser -> /api/mytech -> Node server -> published FastGPT -> normalized JSON -> frontend rendering completed successfully.	PASS
T02	Single-product recommendation	Gaming monitor under ¥1500 returned one valid Monitor recommendation: REDMI G27Q 320Hz at ¥1199.	PASS
T03	One recommendation per request	After prompt fix, a single active product request returned at most one recommendation instead of two cards for the same category.	PASS

ID	Test scenario	Expected/verified behavior	Result
T04	Cheaper follow-up	From REDMI ¥1199, "Give me a cheaper one." returned AOC Q27G40E at ¥879.	PASS
T05	No-cheaper-option handling	After AOC ¥879, another cheaper request correctly returned no recommendation and explained that no cheaper valid gaming monitor existed.	PASS
T06	Alternative follow-up	After AOC ¥879, "Give me an alternative." returned a different valid product: REDMI G27Q 320Hz at ¥1199.	PASS
T07	More-expensive + hard budget	From REDMI ¥1199 with a ¥1500 budget, a more-expensive request did not return the over-budget LG model and correctly reported no valid higher-priced option.	PASS
T08	Multi-product request	Gaming monitor under ¥1500 + gaming mouse under ¥300 returned one Monitor and one Mouse recommendation with separate budgets: REDMI ¥1199 + Logitech G304 ¥229.	PASS
T09	Partial success	Monitor under ¥1500 + gaming laptop under ¥6000 preserved the valid Monitor recommendation and reported no valid laptop match without exceeding budget.	PASS
T10	Clarification	"I need a laptop for gaming." asked for the missing budget instead of recommending prematurely.	PASS
T11	Multi-turn completion	After clarification, "My budget is ¥6000." preserved Laptop + gaming context and returned a no-match explanation without re-asking known information.	PASS

ID	Test scenario	Expected/verified behavior	Result
T12	Unsupported category	Camera request was routed to the unsupported response; no product card was returned.	PASS
T13	Unrelated request	Essay-writing request was routed to the unrelated response; no product card was returned.	PASS
T14	Restart state reset	After Restart, "Give me a cheaper one." did not reuse the previous product context and asked for the product category.	PASS
T15	Help intent / capability starter	"What can you help me with?" returned a capability summary instead of the unrelated fallback.	PASS
T16	Unsure-device starter	"I'm not sure which device suits my needs." triggered clarification and asked for the main purpose/need.	PASS
T17	Greeting intent	"Hi hi" was classified as help and returned the MyTech capability response.	PASS
T18	macOS local execution	Server started successfully with .env loaded; MyTech worked through the local browser.	PASS
T19	Windows local execution	Same package ran successfully on Windows with the expected UI and FastGPT connection.	PASS

4. Key Behavioral Findings

Hard-budget protection

The system consistently avoided presenting over-budget products as valid recommendations. This was directly exercised in the more-expensive follow-up and no-match laptop cases.

Stateful follow-ups

Cheaper, alternative, and more-expensive requests used the previous valid recommendation and active budget correctly during the recorded tests.

Multi-product isolation

Separate category/budget/usage requests were handled independently. Partial success preserved valid recommendations while explaining unmatched requests.

Unified terminal responses

Clarification, valid recommendation, validation failure, unsupported, unrelated, and help branches were normalized to the same top-level JSON response structure before frontend rendering.

Restart isolation

Restart rotated the FastGPT chatId and reset local state, preventing previous context from leaking into a new conversation.

5. Final Read-Only QA Findings

A final code-level QA confirmed the following implementation properties:

- FastGPT credentials are read server-side from environment variables; the API key is not sent to the browser.
- No Gemini endpoint, Gemini environment variable, model-name check, or Gemini adapter remained in the active v2 implementation.
- The frontend uses same-origin API calls and the Node server is stateless across users.
- chatId is generated per browser session/tab; Restart rotates it.
- The nine supported categories and authoritative 27-product dataset were preserved.
- Editorial Clean layout, product-card renderer, keyboard interactions, responsive behavior, and accessibility features were preserved.
- HTTP failures, timeouts, malformed responses, and missing FastGPT content have safe error handling paths.

Non-blocking observations. The “Online / FastGPT Ready” indicator is configuration-based rather than a live connectivity probe. In addition, the server/browser validators do not independently enforce every semantic invariant (for example, duplicate category recommendations) because the published workflow already prevents those cases. These items were judged non-blocking for the classroom demo.

6. DeepSeek Model Switch - Evidence Boundary

Important: The full functional regression listed in Section 3 was completed before the later switch of the FastGPT model to DeepSeek. The workflow contract, routing, variables, validation, and frontend/backend integration were not intentionally changed by the model switch.

After the model switch, targeted spot checks were performed for the updated intent-routing behavior, including:

- Help intent: “What can you help me with?” - PASS
- Unsure-device clarification: “I’m not sure which device suits my needs.” - PASS
- Greeting intent: “Hi hi” - PASS

Not claimed: A complete end-to-end re-run of every test case after the DeepSeek model switch was not performed, primarily to conserve the remaining FastGPT/MAAS AI-point quota. This report therefore distinguishes the earlier full regression from the later targeted post-switch checks.

7. Remaining Optional Checks

- Full regression after the DeepSeek switch if sufficient quota becomes available.
- Live 60-second timeout exercise (the code path was reviewed but not intentionally forced during the manual tests).
- Extreme responsive cases such as short landscape viewports and high browser zoom.
- Production deployment smoke test after the temporary online test URL is created.
- Optional stronger validator rules for duplicate categories or recommendations outside the active request.

8. Conclusion

Final recorded status: READY FOR CLASSROOM SUBMISSION / DEMO

The MyTech v2 package demonstrated correct core recommendation behavior, stateful follow-ups, multi-product handling, clarification, scope routing, Restart behavior, product-card rendering, and local operation on both macOS and Windows. The identified QA observations are non-blocking for the intended classroom use.